

论基于架构的软件设计方法（ABSD）及应用

基于架构的软件设计（Architecture-Based Software Design, ABSD）方法以构成软件架构的商业、质量和功能需求等要素来驱动整个软件开发过程。ABSD 是一个自顶向下，递归细化的软件开发方法，它以软件系统功能的分解为基础，通过选择架构风格实现质量和商业需求，并强调在架构设计过程中使用软件架构模板。采用 ABSD 方法，设计活动可以从项目总体功能框架明确后就开始，因此该方法特别适用于开发一些不能预先决定所有需求的软件系统，如软件产品线系统或长生命周期系统等，也可为需求不能在短时间内明确的软件项目提供指导。

请围绕“基于架构的软件开发方法及应用”论题，依次从以下三个方面进行论述。

1. 概要叙述你参与开发的、采用 ABSD 方法的软件项目以及你在其中所承担的主要工作。
2. 结合项目实际，详细说明采用 ABSD 方法进行软件开发时，需要经历哪些开发阶段？每个阶段包括哪些主要活动？
3. 阐述你在软件开发的过程中都遇到了哪些实际问题及解决方法。

范例

摘要部分：

年月，某集团公司开始了**系统的开发，该系统主要实现**等功能。主要包括**等主要功能模块。我在该系统中担任系统架构师，主要负责该系统的架构设计工作。本文以该系统为例，主要论述了基于架构的软件设计方法在该系统中的具体应用。在架构需求阶段，以构件图、包图、类图和对象图描述系统结构，为系统整体的上层结构进行建模；在架构复审阶段，采用架构权衡分析法来对架构设计方案进行复审，以应对评估质量属性的满足程度问题；在架构演化阶段，采用需求变动管理工具对收集的需求进行归类，以应对需求变动管理问题。经过多轮迭代演化，本系统最终顺利上线，并运行稳定，获得用户一致好评。

【注意：实际写作中相关项目情况应介绍清楚，摘要字数（包括标点符号）一般写到 300 到 320 字】

正文部分：

某集团公司**为了应对***，因此**研发**系统。【项目背景内容可分 2 段写，第 1 段简要说明下项目来龙去脉】

该项目在**年**月正式启动，旨在通过**，优化**。我在该项目中担任架构师，并负责整体系统的架构设计工作。本系统主要由**等主要模块构成。此外，该系统能**，通过如**得到**。**从而最终实现加强某集团在银行业的竞争力的目的。【第 2 段对系统整体情况进行细致介绍，项目背景第 1、2 段内容可以写到 400 到 450 字】

基于架构的软件设计方法，简称 ABSD 方法，主要包含了架构需求、架构设计、架构文档化、架构复

审、架构实现和架构演化六个阶段。其中，架构需求主要包含了需求获取、标识构件、需求评审等活动。架构设计，主要包括提出架构模型、映射构件、分析构件相互作用、产生架构、设计评审等活动。架构文档化，用于把架构设计的成果进行分析与整理并进行文档化。架构复审，用于对架构设计进行复审并标识其中潜在的风险、缺陷和错误。架构实现，包括了架构分析与设计、构件实现、构件组装、系统测试等活动。架构演化，主要活动包括需求变化归类、架构演化计划、构件变动、更新构件的相互作用、构件组装与测试、技术评审等活动。

考虑项目建立初期的需求不稳定，且后续的需要应对大量的新需要，我们决定使用 ABSD 方法来对系统进行构建与演进。下面将着重描述 ABSD 方法在本项目应用过程中所遇到的问题和采取的应对方案。

一、架构需求阶段，以构件图、包图、类图和对象图描述系统结构

为了应对架构需求阶段描述系统架构结构的问题，我们在标识构件活动中，使用构件图、类图等来对系统的整体结构进行建模，实现了通过标识构件活动描述架构结构的目的。首先，我们根据获取的需求以类图和包图的形式为系统的下层结构进行建模。若某个构件的业务构成较为复杂，则以对象图进行辅助说明。然后，我们对这些类图和包图进行分组，从而拟定构件的边界。接着，我们安排项目负责人基于上一步得出的类图和包图的分组，并以需求获取活动得到的会议记录和用例图作为辅助，使用构件图设计出需要的构件，再使用这些构件为系统整体的上层结构进行建模。最后，我们使用《需求构件关系表》标识并记录构件与需求之间的关系，用以指导后续的工作。通过这个方案，我们在标识构件活动中，使用构件图、包图、类图等结构更为清晰的视图为系统的整体结构建立了一个更为稳定的基线，从而达到了提升需求到系统设计的转化效率的效果。

二、架构复审阶段，使用 ATAM 对架构的质量属性进行复审

为了应对对架构设计方案的质量属性满足程度进行评估的问题，我们在架构复审阶段，使用架构权衡分析法（简称 ATAM）来对文档化后的设计方案进行复审，从而实现了确定非功能性需求在本系统的架构设计中是否得到满足的效果。首先，考虑到参与 ATAM 会议的与会人员来自于不同领域的部门，为了便于会议的进行，我们在 ATAM 的描述和介绍阶段中，为相关人员提供 ATAM 评估方法、系统业务动机和架构整体设计作为知识基础。然后，我们在 ATAM 的调查和分析阶段中，使用基于质量属性的评估方法（如建立质量效用树等），对评估系统架构满足非功能需求的情况进行评估。接着，我们在 ATAM 的测试阶段中，基于场景的验证方案来对上一步评估结果进行验证，从而通过质量场景和场景的优先级对架构方案进行调整。通过这个方案，我们在架构复审阶段中，使用 ATAM 来为架构设计方案的质量属性进行评估，从而确定非功能需求在架构设计中的满足程度并通过质量场景调整架构方案的目的。

三、架构演化阶段，对需求变化进行归类

为了应对架构演化阶段的需求变动管理问题，我们在需求变化归类活动中，使用数个工具对收集的需求变动进行整理与分类，实现了对需求变动与构件之间的关系进行管理的目的。首先，我们把新的需求变动登记到《需求管理列表》中，并通过该表获得新的需求变动与旧需求的对应关系。而当某个新的需求变

动和旧需求建立起关系时，我们可以通过上次迭代生成的《需求构件关系表》和《开发工作登记表》，推导出新需求变动与系统已有构件及这些构件的开发人员的对应关系。然后，把新需求与系统中已有构件和候选处理人员的对应关系回填到《需求功能关系表》中。而当某个新需求的需求点和旧需求无法建立起关系时，我们直接把新需求写到《需求构件关系表》中并标记为新增。然后我们再通过召开相关干系人的会议，议定每个变动的实现优先级和跟进人员。通过这个方案，我们不但达到了对需求变动与构件之间的关系进行管理的目的，还能划定较适合处理这些需求变动的候选人名单以指导后续工作之用。

由于使用了以上 ABSD 方法来对项目进行开发与演进，该系统改进工作十分顺利，经过多轮迭代演化，本系统最终于**年**月顺利上线，并运行稳定，用户普遍反馈功能可以满足工作需要且使用体验良好。实践证明 ABSD 方法适用于该项目的变动管理。

然而，我们在应用 ABSD 方法的过程中还存在着一些不足，如我们发现因人手更替，导致一些构件的跟进人员产生变动，这将会产生额外的学习成本。因此，我们决定在后续的演化中，每个构件都需要维护一份更详尽易懂的文档，并在技术评审阶段对本轮迭代中的这些文档进行审核来缓解这个问题。通过本次项目，我认识到紧贴业务需求、数据和开发小组的构成情况来选择具体的软件设计方法将会大大地影响一个项目的演进能力。而在选定软件设计方法后并不意味着结束，通过对方法的细节进行不断改进，将是项目成功的关键。

论软件系统架构风格

系统架构风格（System Architecture Style）是描述某一特定应用领域中系统组织方式的惯用模式。架构风格定义了一个词汇表和一组约束，词汇表中包含一些构件和连接件类型，而这组约束指出系统是如何将这些构件和连接件组合起来的。软件系统架构风格反映了领域中众多软件系统所共有的结构和语义特性，并指导如何将各个模块和子系统有效地组织成一个完整的系统。软件系统架构风格的共有部分可以使得不同系统共享同一个实现代码，系统能够按照常用的、规范化的方式来组织，便于不同设计者很容易地理解系统架构。

请以“软件系统架构风格”论题，依次从以下三个方面进行论述：

1. 概要叙述你参与分析和开发的软件系统开发项目以及你所担任的主要工作。
2. 分析软件系统开发中常用的软件系统架构风格有哪些？详细阐述每种风格的具体含义。
3. 详细说明在你所参与的软件系统开发项目中，采用了哪种软件系统架构风格，具体实施效果如何。

范例

摘要部分：

本人于**年**月参与**项目，该系统为***，在该项目组中我担任系统架构师岗位，主要负责系统整体架构设计。本文以该车联网项目为例，主要讨论了软件架构风格在该项目中的具体应用。底层架构风格我们采用了虚拟机风格中的解释器，因该公交共有几十种不同的数据协议，使用解释器风格可以满足整车数据协议兼容性需求；中间层关于应用层的数据流转采用了独立构件风格中的隐式调用，以减低系统间耦合度、简化软件架构，提高可修改性方面的架构属性；应用系统层采用了 B/S 的架构风格，统一解决公交行业性难题“实施推广难、维护难”问题。最终项目成功上线，获得用户一致好评。

【注意：实际写作中相关项目情况应介绍清楚，摘要字数（包括标点符号）一般写到 300 到 320 字】

正文部分：

随着**，**集团自**年**月起****，规划***。【项目背景内容可分 2 段写，第 1 段简要说明下项目来龙去脉】

年月我司**委托建设***项目。本项目****，我在项目中担任系统架构师职务，主要负责系统整体架构设计，该系统主要***，主要功能模块包括**。【第 2 段对系统整体情况进行细致介绍，项目背景第 1、2 段内容可以写到 400 到 450 字】

在架构工作开始阶段，我们便意识到，架构风格是一组设计原则，是能够提供抽象框架模式，可以为我们的项目提供通用解决方案的，这种能够极大提高软件设计的重用的方法加快我们的建设进程，因此在

我司总工程师的建议下，我们使用了虚拟机风格、独立构件风格以及 B/S 架构风格这三种较常用风格。虚拟机风格中的解释器架构风格能够提供灵活的解析引擎，这类风格非常适用于复杂流程的处理。独立构件风格包括进程通讯风格与隐式调用风格，我们为了简化架构复杂度采用了隐式调用风格，通过消息订阅和发布控制系统间信息交互，不仅能减低系统耦合度，而且还提高架构的可修改性。B/S 架构风格是基于浏览器和服务器的软件架构，它主要使用 http 协议进行通信和交互，简化客户端的工作，最终减低了系统推广和维护的难度，以下正文将重点描述架构风格的实施过程和效果。

底层架构我们使用解释器风格来满足整车数据协议兼容性需求。解释器风格是虚拟机风格中的一种，具备良好的灵活性，在本项目中我们的架构设计需要兼容好 86 种不同 can 数据协议，一般来说这种软件编写难度非常高，代码维护难度压力也很大，因此这个解释器的设计任务便很明确了，软件设计需要高度抽象、协议的适配由配置文件来承担。具体的做法如下，我们对各个车厂的 can 数据结构进行了高度抽象，由于 can 数据由很多数据帧组成，每个数据帧容量固定并且标识和数据有明确规定，因此我们将 can 协议中的 ID 和数据进行关系建模，将整体协议标识作为一个根节点，以 canid 作为根节点下的叶子节点，使用 XML 的数据结构映射成了有整车协议链-数据帧-数据字节-数据位这 4 层的数据结构，核心的代码采用 jdom.jar 与 java 的反射机制动态生成 java 对象，搭建一套可以基于可变模板的解释器，协议模板的产生可以由公交公司提供的 excel 协议文档进行转换得到，解释器支持协议模板热部署，这种可以将透传二进制数据直接映射成 java 的可序列化对象，将数据协议的复杂度简化，后期数据协议更改不会对软件产生影响，仅仅更改协议模板文件即可，最终我们使用了 86 个协议描述文件便兼容了这些复杂的 can 数据协议，规避了 can 数据巨大差异带来的技术风险。

中间层我们使用独立构件风格中的隐式调用来简化构件间的交互复杂度，降低系统耦合度。主要的实现手段是，我们采用了一个开源的消息中间件作为连接构件，这个构件是 apache 基金会下的核心开源项目 activemq，它是一款消息服务器，其性能和稳定性久经考验。由上文提到的解释器解析出对象化数据经过 activemq 分发到各个订阅此消息的应用系统，这些应用系统包括运营指挥调度、自动化机护、新能源电池安全监控等，这种多 web 应用的情况非常适合采用消息发布与消息订阅的机制，能够有效解决耦合问题，我们在编码的过程中发现只要采用这种风格的 web 应用，整个迭代过程效率极高，错误率降低，而且我们使用的 spring 框架，消息队列的管理完全基于配置，清晰简单，维护性良好，例如整车安全主题、运营调度主题、机护维修主题等消息队列分类清晰，可以随时修改其结构也能够随时增其他主题的消息队列，不同的 web 系统监听的队列也可以随时变换组合，基于消息中间件的架构设计能够让系统的构件化思路得到良好实施，总体来说这种架构风格带来了非常清晰的数据流转架构，简化了编码难度，减低本项目的二次开发的难度。

应用系统层我们主要采用 B/S 的架构风格，主要用于解决公交推广难、维护难的问题。公交行业有一个明显的特点，公交子公司分布在全市各个地区，路途很远，且都是内网通讯，车联网络也是走的 APN 专网，一般是无法远程支持的，这给我们的系统推广以及后期维护带来了很大难题，我们可以想象如果使用

C/S 架构，更新客户端一旦遇到问题很可能需要全市各个站点跑一遍。这让我们在系统推广和维护方面面临较大压力。我们采用的 B/S 架构风格能够解决这个难题，并充分考量了现在相关技术的成熟度，例如现在的 html5 完全能够实现以前客户端的功能，项目中我们使用了大量的前端缓存技术与 websocket 技术，能够满足公交用户实时性交互等需求。这种风格中页面和逻辑处理存储在 web 服务器上，维护和软件升级只要更新服务器端即可，及时生效，用户体验较好，例如界面上需要优化，改一下 Javascript 脚本或者 CSS 文件就可以马上看到效果了。

项目于**年**月完成验收，这 1 年内共经历了 2 次大批量新购公交车辆接入，这几次接入过程平稳顺利，其中协议解释器软件性能没有出现过大问题，消息中间件的性能经过多次调优吞吐量也接近了硬盘 IO 极限，满足当前的消息交互总量，另外由于我们的项目多次在紧急状态下能够快速适应 can 协议变动，得到过业主的邮件表扬。除了业主机房几次突发性的网络故障外，项目至今还未有重大的生产事故，项目组现在留 1 个开发人员和 1 个售后在维护，系统的维护量是可控的，系统运行也比较稳定。

不足之处有两个方面，第一在架构设计的过程中我们忽略 PC 配置，个别 PC 因为需要兼容老的应用软件不允许系统升级，这些电脑系统老旧，其浏览器不支持 html5，导致了系统推广障碍。第二在系统容灾方面还有待改善。针对第一种问题，我们通过技术研讨会说服业主新购 PC，采用两台机器同时使用方式解决。针对第二种问题我方采用了服务器冗余和心跳监测等策略，在一台服务暂停的情况下，另外一台服务接管，以增加可用性。

论数据访问层设计技术及其应用

在信息系统的开发与建设中，分层设计是一种常见的架构设计方法，区分层次的目的是为了实现“高内聚低耦合”的思想。分层设计能有效简化系统复杂性，使设计结构清晰，便于提高复用能力和产品维护能力。一种常见的层次划分模型是将信息系统分为表现层、业务逻辑层和数据访问层。信息系统一般以数据为中心，数据访问层的设计是系统设计中的重要内容。数据访问层需要针对需求，提供对数据源读写的访问接口；在保障性能的前提下，数据访问层应具有良好的封装性、可移植性，以及数据库无关性。

请围绕“论数据访问层设计技术及其应用”论题，依次从以下三个方面进行论述。

1. 概要叙述你参与管理和开发的与数据访问层设计有关的软件项目，以及你在其中所担任的主要工作。
2. 详细论述常见的数据访问层设计技术及其所包含的主要内容。
3. 结合你参与管理和开发的实际项目，具体说明采用了哪种数据访问层设计技术，并叙述具体实施过程以及应用效果。

范例

摘要部分：

***年**月，我司承接了**系统的项目，该系统主要实现**，主要包括**等功能模块。我在该项目中担任技术负责人，主要负责该项目技术方案的制定及技术指导工作。本文以**系统为例，主要论述了数据访问层技术在该项目的具体应用。首先在技术选型上，综合需求层面及技术特点的考虑，确定了 Mybatis 技术；接着通过导入资源包、配置核心文件及配置映射文件三大步骤来具体进行数据访问层的开发；最后针对业务执行层面、一二级缓存的配置及数据量大的查询等方面进行性能调优。通过以上技术应用，项目最终顺利上线，达成客户期望，获得用户一致认可。

【注意：实际写作中相关项目情况应介绍清楚，摘要字数（包括标点符号）一般写到 300 到 320 字】

正文部分：

由于**，**支持**，**主导研发**系统。该系统将**进行了严谨精准的管控。【项目背景内容可分 2 段写，第 1 段简要说明下项目来龙去脉】

年**月，我司承接了**系统的建设。**系统作为**，包括**等十几种任务管理模块。主要实现，对**。**系统拥有**，且存储**。因系统**，因此如何使用数据访问层技术来满足本项目的功能及性能需求成了我们考虑的技术问题之一。【第 2 段对系统整体情况进行细致介绍，项目背景第 1、2 段内容可以写到 400 到 450 字】

数据访问层技术也称持久层技术，主要负责将系统数据通过调用数据库的 SQL 语句对数据进行读取和存储。java 语言的 JDK 中自带一种数据访问层技术——JDBC。市面上主流的数据库均实现了 JDBC 中

所定义的对数据库访问的各类接口，因此 JDBC 能够较为高效的完成对各类数据库的数据存取操作。但是，单纯的使用 JDBC 进行开发，效率低下的同时会增加人力成本的支出。基于这种情况，Hibernate 和 Mybatis 技术应运而生。Hibernate 是一个对 JDBC 进行轻量级封装的全自动框架，能够实现数据库道 POJO 的完整映射，开发人员无需编写复杂的 SQL，仅需对 POJO 进行操作便可同步处理数据库中的数据。而 Mybatis 相对 Hibernate 而言则是一个半自动的框架，可以支持自定义的 SQL 语句的操作，相对于复杂的数据查询操作支持性更佳。

针对 SPS 项目，如何应用数据访问层技术，是 SPS 项目组所关注的问题。总体来讲，我们将这个问题分成了三个阶段来处理，分别是技术选型、技术应用以及性能调优。

技术选型

从 SPS 的整体层次上来讲，为了保障 SPS 系统的处理拣料数据的效率，我们将九成以上的事务处理放到了数据库层面进行执行，也就是通过数据库层面的存储过程来完成事务的处理，因为数据库本身的事务处理效率比起 Spring 框架中的事务机制要更加高效，且彼此间的网络开销会更低。而作为上层的 JavaWeb 工程，主要是为用户提供对应的查询检索机制，主要包括两大块：其一，将每一种拣料任务的完成进度为用户提供查询的接口；其二，为用户提供有关拣料任务的各种类型的复杂报表，报表的关联性查询较多。Hibernate 本身的优势在于对数据的更新处理可通过对 POJO 的处理直接映射到对应的数据表/视图中，由于 SPS 系统的事务处理并不在 Java 层面，显然在这个系统中作用不大；另外 Hibernate 是一个全自动框架，对自定义的 SQL 查询支持性并不好；而 Mybatis 相对能够很好的支持自定义 SQL 的开发，对于 SPS 系统而言，不仅能够很好的支持各种简单对象的自定义查询，在复杂报表的开发支持上较 Hibernate 更加强大，能够显著的提升开发效率。因此，从 SPS 系统的需求层面及两种数据访问技术的特点上考虑，Mybatis 更适合 SPS 系统。

技术应用

SPS 系统的开发根据拣料指令业务类型，一共分成了十多个 JavaWeb 工程，每一个 Web 工程专门负责处理一种拣料指令。所以，在应用 Mybatis 进行数据访问层开发时，所有的工程都将引用 Mybatis 的资源 jar 包。在应用过程中，主要包括三大步骤。首先将 Mybatis 的 jar 包资源，导入到一个公共目录中，并将该路径通过 IDE 开发集成环境工具设置到公共资源库中为各个工程包所引用。其次，为所有的工程包配置 Mybatis 的一个名为 sqlMapConfig.xml 的核心配置文件，主要包括整个系统运行时的公共配置信息，例如数据库的基本配置信息（数据库连接路径、驱动、账号及密码）。然后，针对每个模块的开发为其实体类单独配置 Java 对象；例如 SPS 系统用户要查询来自 APA 系统中的计划指令的信息，其实体类为 PlanOrderVO，对应的数据访问接口文件为 PlanOrderMapper.java，该接口文件内定义了一系列的查询计划指令的方法，则在开发过程中同样需要配置一个名为 PlanOrderMapper.xml 的映射文件；映射文件中定义的每一个查询语句皆与接口文件的方法保持一致，且通过<resultMap>标签来定义 Java 对象及其属性与数据库对象及其字段之间的映射关系，同时将自定义的 SQL 语句在该配置文件中完成。

性能调优

从整个系统的角度上讲，性能涉及到方方面面，但从数据访问层技术的应用上看，调优主要包括以下几个方面。首先是业务执行层面的确定。众所周知，事务级操作在数据库层面比在 Java 层面要快得多。因此，对于复杂的事务处理，乃至复杂的报表查询，都定义到了数据库层面（存储过程、视图），Java 层面只需要负责传参和接受结果即可。其次，对于 Mybatis 里面的相关设置的应用，也决定了系统处理数据的效率，最典型的，就是一二级缓存的配置。对 Mybatis 而言，如果配置了缓存，系统会优先在一二级缓存中寻找目标数据，当在一二级缓存中找不到数据时，才会到数据库中进行查询。但是缓存的配置因为缓存的有限容量也存在使用上的讲究。根据二八定律，我们将最常用的上月数据统计报表结果会放入缓存中，避免系统在数据库层面对百万级的数据进行多次的复杂关联查询。最后，对于数据量太大的查询，SQL 语句的写法本身也是决定着查询效率的因素。例如，数据库执行 SQL 语句的过滤条件时，是从后往前执行，因此，如果某个条件能够最大限度的降低数据范围，那么这个条件在 SQL 语句中应该放在后面。

通过以上技术的应用，项目历经**，**系统最终于**年**月顺利上线，目前系统运行稳定，获得了用户一致认可。无论是从业务执行层面，还是在面临数据量大的查询，系统的稳定性都达到了预期。

但在该系统实施过程中，其实也发现了 Mybatis 技术相对于 Hibernate 技术的不足。如 Hibernate 拥有可通过配置方言属性（Dialect）屏蔽数据库类型的特点，让用户不用考虑不同数据库的操作方式，但 Mybatis 因为要支持灵活的 SQL 语句，导致无法支持方言。当某天系统数据库需要进行切换时，对于 Mybatis 而言是一个重大问题，届时需要将所有涉及到与数据库的对接写法全部维护，无疑增加了维护的工作量。要真正解决这个问题，恐怕需要从更高的层面，建立统一的数据库标准后才能实现。当然，这也是我们在下一代技术应用中需要去完善的地方。

论软件系统架构评估

对于软件系统，尤其是大规模的复杂软件系统来说，软件的系统架构对于确保最终系统的质量具有十分重要的意义，不恰当的系统架构将给项目开发带来高昂的代价和难以避免的灾难。对一个系统架构进行评估，是为了：分析现有架构存在的潜在风险，检验设计中提出的质量需求，在系统被构建之前分析现有系统架构对于系统质量的影响，提出系统架构的改进方案。架构评估是软件开发过程中的重要环节。

请围绕“论软件系统架构评估”论题，依次从以下三个方面进行论述。

1. 概要叙述你所参与架构评估的软件系统，以及在评估过程中所担任的主要工作。
2. 分析软件系统架构评估中所普遍关注的质量属性有哪些？详细阐述每种质量属性的具体含义。
3. 详细说明你所参与的软件系统架构评估中，采用了哪种评估方法，具体实施过程和效果如何。

范例

摘要部分：

年月，我参与了**项目的开发，该项目主要功能是**。我在该项目担任系统架构师，主要负责系统整体架构设计及评估等工作。本文以**项目为例，主要论述了软件系统架构评估在本项目中的具体运用。在描述介绍阶段，组织项目团队共同学习 ATAM，促进甲乙双方达成架构评估的共识；在调查分析阶段，建立质量属性效用树，完成各场景的风险点、敏感点和权衡点分析；在测试评估阶段，进行场景的优先级排序，输出评估报告。通过以上技术的使用，使得系统开发的质量得到了保证，为后续项目的顺利实施提供了有力的支撑，最终项目于**年**月正式上线，获得甲方各级管理层的一致好评。

【注意：实际写作中相关项目情况应介绍清楚，摘要字数（包括标点符号）一般写到 300 到 320 字】

正文部分：

随着**的速度发展，**多次出现**，造成**。【项目背景内容可分 2 段写，第 1 段简要说明下项目来龙去脉】

年月，**企业开始了**项目的开发，该系统主要整合了**等子系统，以此解决**等问题。从而支持**满足**多元化需求。该**系统将**整合为**系统。比如**门店的**涉及**等功能模块。项目总投入**万，我在该项目中担任系统架构师，主要负责系统整体架构设计及评估等工作。团队成员**余人，涉及**等方面专家，历时**个月，于**年**月正式上线。【第 2 段对系统整体情况进行细致介绍，项目背景第 1、2 段内容可以写到 400 到 450 字】

项目范围广、系统复杂、涉及各种异构系统整合，因此选择一种合适的架构评估方法显得非常重要。架构所关注的质量属性主要包括性能、可用性、可修改性、安全性等。性能是指系统的响应能力，比如在某段时间内系统所能处理的事件的个数。可用性是系统能够正常运行的时间比例，经常用两次故障之间的

时间长度来表示。安全性是系统在向合法用户提供服务的同时能够阻止非授权用户使用的企图或者拒绝服务的能力。可修改性是指能够快速地对系统性能价格比进行变更的能力，通常以某些具体的变更为基准，通过考察这些变更的代价，来衡量可修改性。常用的架构评估方法有：基于问卷调查的评估方式、基于场景的评估方式和基于度量的评估方式。

**项目在实施企业应用集成时，为了解决 30 余个子系统的集成问题，提高架构的稳定性，主要采用了基于场景的架构权衡分析法 ATAM。下文具体从描述介绍阶段、调查分析阶段、测试评估阶段三个方面来进行论述。

1、描述介绍阶段。

通过组建项目评估小组，组织项目团队共同学习 ATAM，促进甲乙双方达成架构评估的共识。甲方已有 30 余个系统，涉及干系人较多，需要统一的架构评估方法达成共识、保证系统质量。首先根据项目需要成立了项目评估小组，主要成员包括评估小组负责人、项目决策者、架构设计师、用户、开发人员、测试人员、系统部署人员等项目干系人。然后组织召开项目例会，介绍 ATAM 方法，它是一种基于场景的软件架构评估方法，对系统的多个质量属性基于场景进行评估。通过该评估确认系统存在的风险，并检查各自的非功能性需求是否满足需求。该系统的目的和商业动机，主要是通过新零售 iStore 项目建设，完成业务中台、数据中台、AIOT 中台三大系统，支持堂吃、外卖、零售业务的快速增长。最后安排架构组长向项目组成员描述了系统将要采用的 SOA 架构，以及各业务线的领域功能，系统服务端采用 Linux 系统、Java 架构开发。通过以上措施，我们通过 ATAM 的研讨，完成了架构评估的前期准备工作，达成了架构评估的共识。

2、调查分析阶段。

通过收集各项目干系人的质量属性要求，建立质量属性效用树，完成各场景的风险点、敏感点和权衡点分析。智慧门店涉及大量的会员营销活动，需要权衡业务需求方、终端用户、开发人员等项目干系人的质量属性要求。首先组织项目例会，邀请项目组成员发表意见。比如甲方信息中心提出系统要保证其可靠性，系统发生故障必须在 5 分钟内恢复，此优先级最高。而开发人员则提出接口众多，无法预估 BUG 修复的时间。然后组织项目评审，讨论重要的质量属性目标，生成属性质量效应树，例如性能指标的并发数、QPS、平均响应时间。最后分析场景的风险点、敏感点、权衡点。风险点：当第三方聚合支付供应商无法提供服务时，会直接影响业务可用性。敏感点：在线支付信息的加密级别。权衡点：为提高可用性，需要增加集群线路，但集群线路增加，势必影响系统调试的工作量，影响可修改性。通过以上措施，我们建立了质量属性效用树，完成了风险点、敏感点、权衡点分析，提高了架构评估的有效性。

3、测试评估阶段。

通过组织项目例会，进行场景的优先级排序，评估最佳的应对方案，输出评估报告。数据中台、业务中台、AIOT 中台涉及的业务、数据需要全部在线化、迁入云端，如果不能支持高并发，则会导致出现大范围的服务无响应。首先组织评估小组集体讨论，确定了不同场景的优先级。系统的可用性最高，性能其次，

可修改性及安全性优先级较低。主要是通过 Nginx 集群、Tomcat 负载均衡、Redis 集群、MySQL 集群，良好的支持了营销活动的抢红包、秒杀等场景。然后评估服务注册中心、分布式开关的限流、降级的有效性。采用令牌桶算法实现限流策略，控制访问速率以及应对突发性的流量。另外考虑网络的异常情况，增加监测机制，针对超时无响应的情况，采取回滚机制，提高可用性。最后制定了评估报告，结合系统的风险点、敏感点、权衡点和非风险点，递交了架构分析文档、架构场景及优先级、质量属性效应树、风险点决策等文档。通过以上措施，我们实施了 ATAM 架构评估，制定了架构评估报告，提升了架构评估的全面性。

通过使用软件系统架构评估，使得系统开发工作完成得非常顺利，系统开发的质量得到了保证，对后续项目的顺利实施提供了有力的支撑。项目于**年**月完成上线，通过中台系统的订单聚集分发，各项性能指标达到甲方要求，并经受住了 100 万用户参与的双 11 秒杀活动，系统宕机的问题得到解决，营销活动的千人千面也取得了很好的效果，项目获得了甲方管理层的表扬。

虽然项目整体上取得了成功，但具体实施过程中，其中对于 APP 架构评估的重视度不够，没有考虑微信小程序与微信公众号之间的衔接问题。通过堂吃、外卖、商城界面的导航栏统一，优化了小程序和 H5 的访问入口，提高了用户体验。另外由于需求的变化，频繁的升级，影响独立构件的稳定性，通过微服务的部署，提升了独立构件的灵活性。通过这次的评估工作，使我对软件架构的评估工作方方面面都有更深入的理解，规范的、正确的软件架构评估方法会给系统的开发阶段带来效率的大幅提高，基于经过评估的架构开发出来的系统质量有很好的保证，大幅减少的了需求实现的偏差。

论云原生架构及其应用

近年来，随着数字化转型不断深入，科技创新与业务发展不断融合，各行各业正在从大工业时代的固化范式进化成面向创新型组织与灵活型业务的崭新模式。在这一背景下，以容器和微服务架构为代表的云原生技术作为云计算服务的新模式，已经逐渐成为企业持续发展的主流选择。云原生架构是基于云原生技术的一组架构原则和设计模式的集合，旨在将云应用中的非业务代码部分进行最大化剥离，从而让云设施接管应用中原有的大量非功能特性(如弹性、韧性、安全、可观测性、灰度等)，使业务不再有非功能性业务中断困扰的同时，具备轻量、敏捷、高度自动化的特点。云原生架构有利于各组织在公有云、私有云和混合云等新型动态环境中，构建和运行可弹性扩展的应用，其代表技术包括容器、服务网格、微服务、不可变基础设施和声明式 API 等。

请围绕“论云原生架构及其应用”论题，依次从以下三个方面进行论述：

- 1.概要叙述你参与管理和开发的软件项目以及承担的主要工作。
- 2.服务化、弹性、可观测性和自动化是云原生架构的四类设计原则，请简要对这四类设计原则的内涵进行阐述。
- 3.具体阐述你参与管理和开发的项目是如何采用云原生架构的，并且围绕上述四类设计原则，详细论述在项目设计与实现过程中遇到了哪些实际问题，是如何解决的。

范例

摘要部分：

年月，我作为技术负责人参与了某市**平台建设项目，该项目主要包括**，打造**，构建**。本文将以**建设为例，阐述云原生架构在本项目中具体实践。基于云原生弹性原则，解决以往项目中系统资源无法根据需求进行弹性扩容的问题。基于云原生自动化原则，解决项目中应用上线周期长，复杂度高的问题。基于云原生服务化原则，解决以往系统建设中各系统共性功能重复建设的问题。通过云原生架构的成功使用，平台实现资源层和应用层的弹性升缩能力，提升了平台对外提供服务的能力，及项目整体自动化水平。现在平台已经顺利上线运营，取得非常良好社会效益和经济效益。【注意：实际写作中相关项目情况应介绍清楚，摘要字数（包括标点符号）一般写到 300 到 320 字】

正文部分：

年月公司中标我市**项目，项目总投资**万元人民币，项目建设周期**年，我作为技术负责人全程参与了本项目。【项目背景内容可分 2 段写，第 1 段简要说明下项目来龙去脉】

本项目**，项目总体建设内容包括***，为***提供***；采用***，我们基于****技术路线，将平台****。

【第 2 段对系统整体情况进行细致介绍，项目背景第 1、2 段内容可以写到 400 到 450 字】

我们在项目中采用云原生架构，充分体现云原生架构的自动化、服务化、弹性，及可观测性四大原则。自动化原则，体现应用的自动发布，自动化代码检查，及自动化测试等自动流程；服务化原则，体现在云原生架构实现了“微服务，轻应用”，按需对外提供服务；弹性原则主要来源于云计算特有对云资源的弹性伸缩能力；可观测性，通过相关采集系统和监测工具的使用，系统的可观测性进一步加强。接下来，我将基于本项目的实践着重论述自动化原则，服务化原则，及弹性原则。

一、自动化原则。以往的项目中上线一个应用，需要准备环境，搭建服务器，有时因为开发时没考虑上线时的需要，如浏览器及客户端等适配问题，经常导致上线工作周期长，问题多，经常返工。本项目中我们一方面采用了 K8S 来进行容器的编排和镜像的管理，基于容器技术实现了应用的自动化部署，及按需灵活扩容。为了实现自动化上线和自动运维，我在项目倡导了 DevOps 实践，开发人员从写下第一行代码开始，就要考虑相关后期上线及运维的工作，运维人员也在项目一开始就介入其中，使得从编码到上线更加顺畅高效。同时我也引入了相关自动代码检查，代码安全漏洞扫描工具，代替以往的人工走查和桌面检查，我们的测试团队也开发了相关自动测试工具，引入相关云服务商提供的自动压测工具，极大提升了团队的工作效率和项目质量。

二、服务化原则。以往系统建设中，多是竖井式建设，烟囱式系统，小而全，大而全，但工业互联网平台将要开发大量的工业应用，如果还是按照以前方式，一是工作量巨大，二是一定会有大量的重复建设，造成工期和成本的巨大浪费。项目中我们基于 SOA 的架构思想，采用现在流行的“微服务，轻应用”理念，采用 spring cloud+docker 的技术路线，我们将平台沉淀的相关设备管理、生产管理、供应链管理，及工业大数据分析能力拆分解耦，封装为一个个微服务，实现平台整体的“高内聚，低耦合”，这些服务通过微服务网关支撑各类上层工业应用服务建设，及工业 app 开发。通过这种方式，我们一方面实现了共性能力的复用，达到了集约化建设目标，另一方面也将平台能力通过微服务的方式开放给第三方开发者，将平台沉淀的相关数据分析，智能制造，设备管理能力对外赋能，为构建开放共享的区域工业互联网应用生态打下了坚实基础。

三、弹性原则。以往的系统平台建设中，如何按需对项目所需要的存储、计算、网络资源进行动态扩容，以及在面临海量高并发的情况下，如何能够实现应用服务的高可用也是一直没有很好解决的难题。我通过打造两方面的能力，实现在云原生架构的区域工业互联网平台的弹性伸缩能力。为了解决这个问题，我们从三方面着手解决：一是在 IaaS 层通过 VMware 对存储，计算资源进行了虚拟化，构建了统一的虚拟化云资源池，并基于 OpenStack 打造了云资源服务统一管理平台，也就是区域工业云平台，基于 OpenStack 实现了云资源的弹性供给，动态管理，及按需扩容。二是在 PaaS 层通过 K8S，实现根据服务请求数量，实时调整相关微服务数量，使用系统就算在高并发条件下也具备具有非常高的可用性，比如在实际使用中我们设备平台的协议转换服务请求较多时，通过 k8s 及时上线相关备用服务，满足忙时业务需求，当闲时

也可以非常方便的下线相关应用服务，释放系统资源，提升了平台服务层的弹性。

年月项目正式上线运营，目前平台已经为区域内 50 多家工业企业提供上云服务，接入工业设备超 200 台，沉淀工业大数据近 50PB，提供工业微服务近 200 个，打造了工业 app 近 30 个，初步构建了开放共享的区域工业互联网生态，取得了比较好的社会效益和经济效益。

通过项目的实践，我深刻体会到云原生架构不仅是技术创新，更是组织与管理的变革，如果不能建立起开发、运维、及质量保证融合推进的组织架构，没有敏捷开发及快速迭代的文化，云原生架构带来自动化，服务化，弹性，及可观性优势都会大打折扣，流于形式。虽然项目中取得一定成绩，但也暴露了比较多的问题，项目初期由于大量采用开源的新技术和新架构，技术复杂开发难度大，导致了项目推进举步维艰，我一方面引进外部专家，开展多轮全员培训，并招聘相关专业技术人员进入团队；另一方面根据我们项目的实际情况，提出直接按需购买相关云厂商产品服务的方案。通过综合施策，项目得以顺利开展。项目的成功不是我一个人的功劳，在此向参与项目各位成员表示最真诚地感谢。凡是过往，皆为序章，我将继续保持空杯心态，不断学习提升自己的综合素养，继续为祖国的信息化事业贡献自己的绵薄之力。



免费订阅考试资讯



更多备考资料及学习福利
扫码添加希赛网课程顾问了解



免费在线刷题